FACULTAD DE DEFENSA NACIONAL (FADENA)

LICENCIATURA EN CIBERDEFENSA

ASIGNATURA: TECNOLOGÍA OPERATIVA (2026)

GUÍA DE ACTIVIDADES PRÁCTICAS

UNIDAD 1: FUNDAMENTOS FÍSICOS Y HARDWARE DE CONTROL

Herramientas Gratuitas Web y Descargables

ACTIVIDAD PRÁCTICA N° 3

SIMULACIÓN DE SEÑAL 4-20 mA Y DETECCIÓN DE SENSOR SPOOFING

Documento de Cátedra N° 3 — Interacción con el Mundo Físico

1. Datos de la Actividad

Asignatura	Tecnología Operativa
Unidad / Documento	Unidad 1 — Documento de Cátedra N° 3
Duración estimada	75 minutos
Modalidad	Individual
Herramienta principal	Google Colab (https://colab.research.google.com) — Python en la nube
Herramienta alternativa	Node-RED (https://nodered.org) — gratuito, descargable
Tipo de herramienta	Web — Gratuita con cuenta Google (Colab) / Descargable gratuita (Node-RED)
Prerequisito	Lectura del Documento de Cátedra N° 3

2. Objetivo General

Simular mediante programación el comportamiento del estándar industrial 4-20 mA en condiciones normales, de falla física y de ataque de Sensor Spoofing, desarrollando la capacidad de interpretar telemetría analógica y detectar anomalías que puedan indicar un ciberataque activo sobre la integridad de los datos de proceso.

3. Objetivos Específicos

- Calcular y graficar la conversión entre corriente (mA) y valor de proceso según el estándar 4-20 mA.
- Simular el comportamiento de un transmisor en condición normal, falla (cable cortado) y ataque Spoofing.
- Distinguir visualmente mediante gráficos la diferencia entre señal legítima, señal de falla y señal manipulada.
- Relacionar la lectura anómala de un transmisor con el principio de Integridad de la tríada AIC.

4. Herramienta: Google Colab

¿Qué es Google Colab?

Google Colab es un entorno Python gratuito en el navegador, sin instalación. Permite escribir y ejecutar código Python, generar gráficos y analizar datos. Requiere cuenta de Google. Es ampliamente utilizado en ciberseguridad, ciencia de datos e investigación académica. El código se ejecuta en servidores de Google; no se requiere hardware de alta prestación.

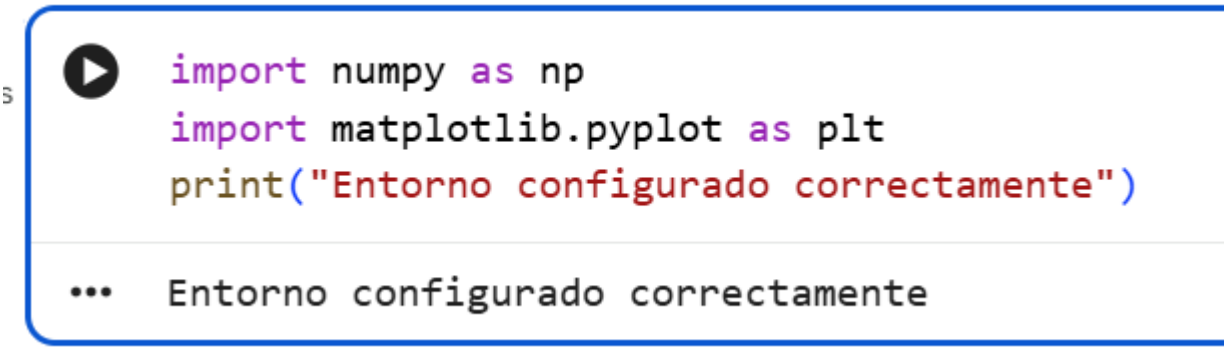
5. Procedimiento Paso a Paso

PARTE A — Configuración del entorno

1 Abrir https://colab.research.google.com e iniciar sesión con cuenta Google. Crear nuevo notebook: "File" → "New notebook". Renombrarlo "TP_Unidad1_4-20mA.ipynb".

2 En la primera celda, escribir y ejecutar (botón ▶ o Shift+Enter): import numpy as np import matplotlib.pyplot as plt print("Entorno configurado correctamente")

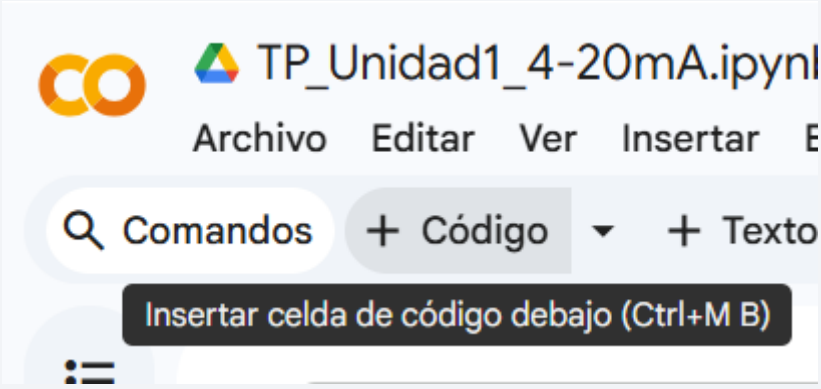
Nota: son tres sentencias y no deben estar en la misma línea.



PARTE B — Función de Conversión 4-20 mA

3 Implementar la función de conversión. Copiar y ejecutar: def ma_to_percent(ma): if ma < 2.0: return None # cable abierto elif ma < 4.0: return "FALLA: fuera de rango" else: return ((ma - 4) / 16) * 100 for test_ma in [0, 4, 8, 12, 16, 20]: print(f"{test_ma:5.1f} mA -> {ma_to_percent(test_ma)}")

Nota: por cada PARTE nueva deberá insertar celda de código debajo con botón de barra de menú <+ Código>.



Nota: En Python, el código dentro de una función o bloque condicional debe estar en líneas nuevas y con la indentación correcta.

```
def ma_to_percent(ma):
    if ma < 2.0:
        return None  # cable abierto
    elif ma < 4.0:
        return "FALLA: fuera de rango"
    else:
        return ((ma - 4) / 16) * 100

for test_ma in [0, 4, 8, 12, 16, 20]:
    print(f"{test_ma:5.1f} mA -> {ma_to_percent(test_ma)}")
```

4

Analizar la salida. Verificar: 0 mA = None (falla), 4 mA = 0%, 12 mA = 50%, 20 mA = 100%. Completar la Tabla de Conversión.

Nota: El resultado muestra la conversión de los valores en miliamperios (mA) a porcentajes, utilizando la función ma_to_percent que definiste. Los valores por debajo de 4.0 mA devuelven None, los de 4.0 mA devuelven 0%, y así sucesivamente hasta 20.0 mA que devuelve 100%.

```
...     0.0 mA -> None
        4.0 mA -> 0.0
        8.0 mA -> 25.0
       12.0 mA -> 50.0
       16.0 mA -> 75.0
       20.0 mA -> 100.0
```

PARTE C — Simulación de los Tres Escenarios

5

Generar las tres señales. Copiar y ejecutar: t = np.linspace(0, 60, 600) # SEÑAL NORMAL: nivel subiendo de 20% a 80% normal = 4 + (16 * (0.2 + 0.6 * (t / 60))) # FALLA FÍSICA: cable cortado a los 30 segundos falla = normal.copy() falla[t > 30] = 0 # 0 mA = cable cortado # SPOOFING: atacante congela el valor en 12 mA spoofing = normal.copy() spoofing[t > 30] = 12 # valor congelado print("Escenarios generados")

Nota: En Python, cada instrucción debe ir en su propia línea y los bloques de código requieren sangría.

```
t = np.linspace(0, 60, 600)

# SEÑAL NORMAL: nivel subiendo de 20% a 80%
normal = 4 + (16 * (0.2 + 0.6 * (t / 60)))

# FALLA FÍSICA: cable cortado a los 30 segundos
falla = normal.copy()
falla[t > 30] = 0  # 0 mA = cable cortado

# SPOOFING: atacante congela el valor en 12 mA
spoofing = normal.copy()
spoofing[t > 30] = 12  # valor congelado

print("Escenarios generados")
```

```
Escenarios generados
```

6 Graficar y comparar: fig, axes = plt.subplots(3, 1, figsize=(12, 10)) fig.suptitle("Estándar 4-20 mA: Comparación de Escenarios", fontsize=14, fontweight="bold") for ax, sig, title, color in zip(axes, [normal, falla, spoofing], ["SEÑAL NORMAL","FALLA FÍSICA (Cable Cortado)","ATAQUE: Sensor Spoofing"], ["green","orange","red"]): ax.plot(t, sig, color=color, linewidth=2) ax.axhline(y=4, color="blue", linestyle="--", alpha=0.5) ax.axvline(x=30, color="black", linestyle=":", alpha=0.7) ax.set_title(title, fontweight="bold", color=color) ax.set_ylabel("Corriente (mA)") ax.set_ylim(-1, 22) ax.grid(True, alpha=0.3) plt.tight_layout() plt.savefig("escenarios_4-20mA.png", dpi=150) plt.show()

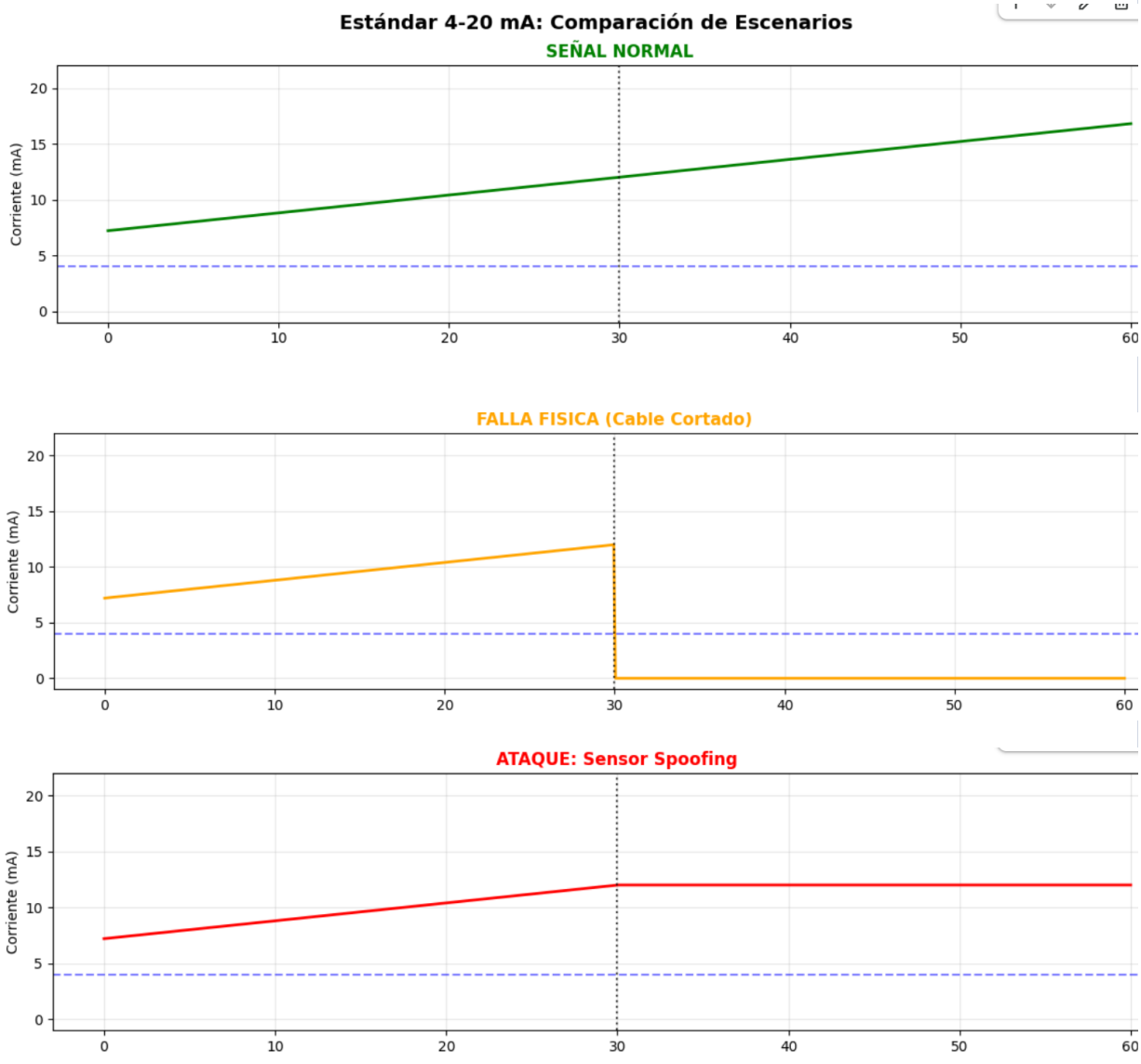
Nota: En Python, cada instrucción debe ir en su propia línea y los bloques de código requieren sangría.

```
fig, axes = plt.subplots(3, 1, figsize=(12, 10))
fig.suptitle("Estándar 4-20 mA: Comparación de Escenarios", fontsize=14, fontweight="bold

for ax, sig, title, color in zip(axes,
    [normal, falla, spoofing],
    ["SEÑAL NORMAL","FALLA FÍSICA (Cable Cortado)","ATAQUE: Sensor Spoofing"],
    ["green","orange","red"]):
    ax.plot(t, sig, color=color, linewidth=2)
    ax.axhline(y=4, color="blue", linestyle="--", alpha=0.5)
    ax.axvline(x=30, color="black", linestyle=":", alpha=0.7)
    ax.set_title(title, fontweight="bold", color=color)
    ax.set_ylabel("Corriente (mA)")
    ax.set_ylim(-1, 22)
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("escenarios_4-20mA.png", dpi=150)
plt.show()
```

Resultados:



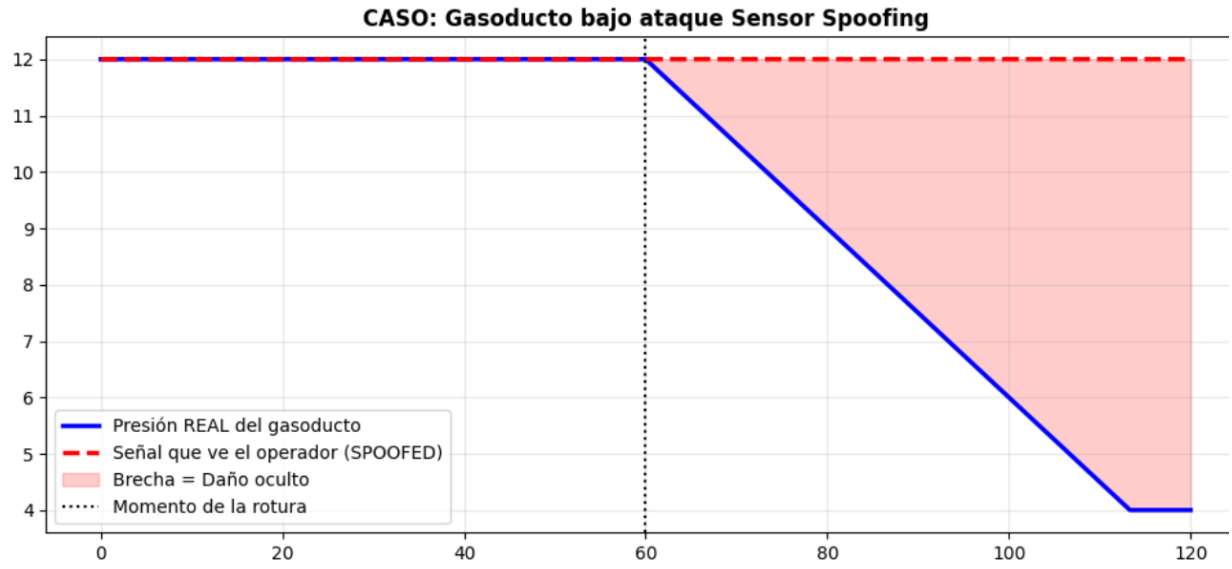
7

ESCENARIO GASODUCTO — Simular el caso práctico del Documento N° 3: t2 = np.linspace(0, 120, 1200) presion_real = np.where(t2 < 60, 12, np.maximum(4, 12 - 0.15*(t2-60))) presion_hmi = np.where(t2 < 60, 12, 12) # congelado por el atacante plt.figure(figsize=(12,5)) plt.plot(t2, presion_real, "b-", lw=2.5, label="Presión REAL del gasoducto") plt.plot(t2, presion_hmi, "r--", lw=2.5, label="Señal que ve el operador (SPOOFED)") plt.fill_between(t2[t2>=60], presion_real[t2>=60], presion_hmi[t2>=60], alpha=0.2, color="red", label="Brecha = Daño oculto") plt.axvline(x=60, color="black", linestyle=":", label="Momento de la rotura") plt.title("CASO: Gasoducto bajo ataque Sensor Spoofing", fontweight="bold") plt.legend(); plt.grid(True, alpha=0.3) plt.savefig("gasoducto_spoofing.png", dpi=150) plt.show()

Nota: si hay varias instrucciones de Python en una sola línea sin una separación adecuada, provocará que Python no puede procesar.

```
t2 = np.linspace(0, 120, 1200)
presion_real = np.where(t2 < 60, 12, np.maximum(4, 12 - 0.15*(t2-60)))
presion_hmi  = np.where(t2 < 60, 12, 12)  # congelado por el atacante
plt.figure(figsize=(12,5))
plt.plot(t2, presion_real, "b-", lw=2.5, label="Presión REAL del gasoducto")
plt.plot(t2, presion_hmi,  "r--", lw=2.5, label="Señal que ve el operador (SPOOFED)")
plt.fill_between(t2[t2>=60], presion_real[t2>=60], presion_hmi[t2>=60], alpha=0.2, color="red", label="Brecha = Daño oculto")
plt.axvline(x=60, color="black", linestyle=":", label="Momento de la rotura")
plt.title("CASO: Gasoducto bajo ataque Sensor Spoofing", fontweight="bold")
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig("gasoducto_spoofing.png", dpi=150)
plt.show()
```

Nota de la salida:
Se ha generado un gráfico titulado "CASO: Gasoducto bajo ataque Sensor Spoofing". Este gráfico visualiza la "Presión REAL del gasoducto" en contraste con la "Señal que ve el operador (SPOOFED)", mostrando la "Brecha = Daño oculto" que surge de un ataque de spoofing. La línea vertical punteada en x=60 marca el "Momento de la rotura".



8

Descargar ambos gráficos (clic derecho → Guardar imagen) para adjuntarlos como evidencia en la entrega.

6. Tabla de Registro de Conversión 4-20 mA

Valor (mA)	% de Proceso	Estado	Significado OT
0 mA			
4 mA			
8 mA			
12 mA			
16 mA			
20 mA			

7. Cuestionario de Cierre

1. Observando el escenario "FALLA FÍSICA", el valor cae a 0 mA a los 30 segundos. ¿Por qué un sistema OT bien diseñado reconoce esto como falla y NO como "proceso en 0%"? ¿Qué acción automática debería tomar el PLC?

2. En el gráfico del caso Gasoducto, la brecha roja representa gas escapando sin que el operador lo sepa. ¿Qué principio de la tríada AIC está siendo vulnerado? ¿Qué mecanismo técnico de detección podría descubrir este ataque aun cuando la señal del transmisor parece normal?

8. Criterios de Evaluación

Criterio de Evaluación	Ponderación	Indicador
Código ejecutado y gráficos generados (evidencia: capturas de pantalla)	25%	Screenshots adjuntos
Tabla de conversión completada sin errores	25%	Valores calculados
Análisis del estándar 4 mA como mecanismo fail-safe (pregunta 1)	25%	Argumentación técnica
Identificación del principio AIC comprometido y mecanismo de detección (pregunta 2)	25%	Aplicación de la tríada

Versión y Créditos

Guía de Actividades Prácticas — Unidad 1 — Versión 1.0 — FADENA — 2026 Asignatura: Tecnología Operativa | Licenciatura en Ciberdefensa Todas las herramientas son gratuitas y de uso educativo libre. Verificar disponibilidad de URLs antes de cada clase.